

VM2 Containerized Reconstruction Plan

Rebuild Ben's four-agent proto-hive from the verified VM2 copy

Prepared for: ETP and Benjamin Goertzel **Date:** 15 August 2026

Executive decision

Use **one pinned shared container image and four separate Compose services**, one service per agent identity: ProtoCosmo, ProtoCosmo2, Protomega, and Protomega2. Each service receives its own read-only configuration, read-write state, secret mount, health check, and activation gate. A single all-agents container is technically feasible but is not recommended: it creates one failure domain, obscures which receiver failed, complicates per-agent rollback, and gives every process access to every secret.

The four-service design is still operationally simple: one Compose project, one image build, one host directory tree, and a small set of operator commands. Docker isolates application dependencies; it must not manage the host firewall, SSH daemon, network interfaces, or provider access rules.

Verified starting point

The VM2 transfer gate completed at 12:40 PDT on 15 August 2026. The frozen 2.425 GB workspace archive matched SHA-256 and exactly 58,538 entries. Per-agent receipts passed: ProtoCosmo 48,719 files / 5,395,328,467 bytes; ProtoCosmo2 7 / 313,248; Protomega 6 / 229,485; Protomega2 5 / 160,685. Directory and secret modes passed. VM2 had no proto-hive containers and no polling marker. Laptop receivers remained authoritative.

This proves transfer integrity, not runtime correctness. The next work must preserve VM2 inertness until image, mounts, offline tests, backup/restore, and activation gates pass.

Target architecture

Component	Design
Image	One immutable, versioned image containing pinned OS packages, OpenClaw/Omega/PeTTa runtimes, agent launcher, offline health probe, and no credentials or mutable state.
Compose	One project with four services. Use profiles or explicit service names so an operator can validate and activate one identity at a time.
State	Four isolated host paths, mounted read-write only into the matching service. Never use an anonymous volume for irreplaceable state.
Secrets	Four mode-0600 host files or Docker secrets, mounted read-only only into the matching service. Secret values never enter the image, Compose file, logs, Git, or command line.
Network	Outbound-only default bridge network. Publish no application ports unless a measured requirement exists. Telegram polling needs outbound HTTPS, not an inbound public port.
Supervision	Docker restart policy plus a host systemd unit that starts the Compose project only after an explicit activation marker exists.
Observability	Per-service health check, bounded logs with rotation, status command, and receipts that avoid secret content.

Suggested host layout:

/opt/proto-hive/

```

compose.yaml      # root-owned, 0644; no secret values
env/              # non-secret per-agent configuration
state/<identity>/  # root-owned, 0700; mounted read-write
secrets/<identity>/ # root-owned, 0700; files 0600
backups/          # encrypted or access-restricted rollback snapshots
receipts/         # hashes, counts, test and activation evidence
ACTIVATE_<identity> # absent until the controlled activation gate

```

Access and lockout prevention

This reconstruction must not modify the host firewall, provider firewall, SSH configuration, routing, or network manager. Containers do not require those changes.

Before every host-level package or service action:

1. Keep the current SSH session open and open a second key-only SSH session as a canary.
2. Capture current SSH reachability and Docker/Compose state without printing credentials.
3. Make only the bounded Docker-related change; never restart SSH or networking.
4. Re-test the second session before closing the first.
5. Prefer provider-console access as an emergency path, but do not alter access rules merely as a precaution.

Docker rules must not expose agent ports. Do not use host networking, privileged containers, Docker socket mounts, broad host filesystem mounts, or capabilities such as `NET_ADMIN`. Use `no-new-privileges`, drop Linux capabilities, run as a non-root UID when compatible, and use a read-only root filesystem with explicit writable state and temporary mounts.

Execution phases

Phase 0 — Freeze and map the copied data

Inventory the verified VM2 trees without changing them. Classify every required path as image input, read-only configuration, per-agent mutable state, secret, ephemeral cache, or excluded material. Record exact runtime versions, source hashes, entrypoint, working directory, environment-key names, schedules, database paths, and expected ownership. Never copy the entire transferred tree into an image.

Gate: Every required file has one mount/build classification; all secret values and unrelated personal data are excluded from the build context; all four activation markers remain absent.

Phase 1 — Build one credential-free image

Create a minimal allowlisted build context from the verified code and dependency manifests. Pin the base image by digest and dependencies by lockfile/version. Build locally on VM2 without starting an identity service. Scan the context and resulting image history for recognizable secrets and unexpected large files. Record image ID and content digest.

Gate: The build is reproducible from an allowlisted context; secret scan passes; no agent container has started; SSH canary still succeeds.

Phase 2 — Assemble four isolated services

Create the host tree and populate each agent's config, state, and secrets from its verified VM2 copy. Preserve modes and ownership. Define four Compose services from the same image, differing only in identity, mounts, command/config, and health check. Use `network_mode: none` for initial tests, no

published ports, bounded log rotation, read-only root filesystem, and service-specific activation-marker checks in the endpoint.

Gate: `docker compose config` validates without disclosing secrets; mount receipts match; a service cannot start without its own marker; no service can read another identity's secret or state.

Phase 3 — Offline and restore validation

Run each service with networking disabled and polling forcibly disabled. Exercise imports, database reads, memory/session discovery, schedule parsing, local tools, and health checks. Create a state backup, restore it into a disposable path, and prove file counts, hashes for immutable artifacts, database integrity, and required modes. Test container restart and host reboot while all activation markers are absent.

Gate: All four offline suites pass; backup/restore passes; reboot leaves all receivers inert; laptop bots remain unchanged.

Phase 4 — Controlled one-at-a-time activation

Decide explicitly whether VM2 will be an independent branch with new Telegram identities or a migration of existing identities. For an independent branch, inject new bot credentials and start with fresh cursors. For an identity migration, stop and verify the matching laptop receiver immediately before final state delta and VM2 activation. Never allow two pollers for one token.

Activate only one service. Confirm exactly one process/container owns that identity, send a fresh human-authored text canary and safe attachment canary, verify reply routing, state persistence, and log hygiene, then restart that service and repeat the canary. Proceed to the next identity only after acceptance.

Gate: For each activated identity: exactly one receiver, fresh canaries pass, restart passes, state advances correctly, and no other identity regresses.

Phase 5 — Reboot, soak, and handover

Reboot VM2 through the normal provider/host mechanism only after all four identities pass. Verify SSH first, then Docker, Compose, service health, one receiver per identity, schedules, disk usage, and canaries. Soak for at least 24 hours with bounded logs and disk alerts. Retain the laptop as a stopped-but-ready rollback target until the soak is accepted.

Gate: Reboot and soak pass with no duplicate receivers, missed routing, state corruption, unbounded logs, or SSH regression; operator handover and receipts are complete.

Rollback

Rollback is per identity. Stop the affected VM2 service, remove its activation marker, verify its receiver count is zero, restore the last accepted state snapshot if needed, then restart the approved prior receiver (laptop or prior VM) and send a fresh canary. Never start the old receiver until the VM2 receiver is proven stopped. Preserve the failed container logs, image digest, state snapshot, and timeline for diagnosis.

If SSH becomes unreliable, do not change firewall or SSH configuration from a partially connected shell. Stop application activation work, retain the open session, use the provider console if available, and diagnose reachability separately from the containers.

Acceptance checklist

- One pinned image digest; four independently controllable services.
- No credentials/state in image layers, Compose source, Git, logs, or process arguments.
- Per-agent state and secret isolation proven; modes and ownership recorded.
- No privileged mode, host networking, Docker socket, broad host mounts, or published ports.
- Offline tests and disposable backup/restore pass before network activation.
- Exactly one Telegram receiver per identity; fresh text and attachment canaries pass.
- Service restart and VM2 reboot pass; SSH remains reachable throughout.
- Laptop rollback remains verified through the soak period.
- Operator commands and evidence are stored under the project record without secret values.

Recommended immediate next action

Do not start a bot yet. Perform Phase 0 on the accepted VM2 trees, then produce the mount manifest and credential-free build allowlist. This is the shortest safe path to a working deployment because it uses the data already present, avoids another transfer, and prevents a hidden token, mutable database, or laptop-only path from being baked into a fragile all-in-one container.

Evidence basis and scope

This plan uses the accepted VM2 transfer evidence recorded in the migration experiment's `RUN.md`, the project task/decision records, and the previously adopted Compose design. It does not claim that the copied bots have yet been built or run successfully on VM2. The separate Omega-agent infrastructure repair and the Iter upgrade are outside this reconstruction gate; upgrades should follow baseline reconstruction so migration failures and semantic/runtime changes remain distinguishable.